POST vs. PUT: HTTP Methods Explained

https://gemini.google.com/app/1faf8a8096538ed9

User prompt: Context: you are a senior web software developer with 200 IQ and 20 years of experience. Your are also an expert in HTTP communication protocol. Prompt: what are the main differences between POST and PUT request methods? When should I use one or other? be concise, technical and strait to the point.

Response: The primary architectural differences between **POST** and **PUT** in HTTP/1.1 and HTTP/3 (RFC 9110) stem from three core traits: **idempotency**, **URI target responsibility**, and **mutation semantics**.

Core Technical Differences

1. Idempotency

- **PUT is idempotent:** Executing N identical PUT requests produces the exact same server state as executing it once ($f(x) = f(f(x))$).
- **POST is non-idempotent:** Executing N identical POST requests creates N distinct resources or triggers N side-effects.

2. Target URI & Resource Identification

- **PUT targets the exact resource URI:** The client controls or knows the resource identifier prior to sending the request.

  ```
  PUT /api/v1/users/usr_98a71b HTTP/1.1
  ```

  (The client dictates that the resource location is `/users/usr_98a71b`.)

- **POST targets a factory or processing endpoint:** The server generates the resource ID and controls the resulting location.

  ```
  POST /api/v1/users HTTP/1.1
  ```

  (The server creates the resource, assigns an ID, and returns a `201 Created` status with `Location: /api/v1/users/usr_98a71b`.)

3. Payload Semantics

- **PUT is a Full Replacement (or Create):** The payload represents the complete enclosed representation of the target resource. Any omitted fields should be treated as `null` or reset to defaults by the server.
- **POST is Data Processing / Sub-resource Creation:** The target resource processes

the request body according to its own semantics (e.g., appending to a log, submitting a form, or initializing a process).

Quick Comparison

Attribute	POST	PUT
Idempotent	No	**Yes**
Safe (Read-Only)	No	No
Target URI	Collection / Processing Endpoint (`/orders`)	Specific Resource (`/orders/1042`)
Primary Intent	Append, process, or trigger action	Replace completely, or create at specific URI
Typical Success Code	`201 Created` or `200 OK`	`200 OK`, `204 No Content`, or `201 Created`
Network Retry Safety	Low (Risk of duplicate side-effects)	**High** (Safe to retry on network timeout)

When to Use Which

Use PUT when:

- **You are replacing a resource entirely:** Updating an existing record where the client sends the entire state representation.
- **You are doing a client-defined "Upsert":** Creating a resource where the client generates the unique key/UUID in advance (`PUT /widgets/uuid-here`).
- **Retry safety matters:** On unstable connections where network retries must guarantee no duplicate records are generated.

Use POST when:

- **The server manages identity:** Creating a child resource where the database auto-increments or assigns the primary key.
- **Executing non-CRUD controller operations:** Triggering workflows, complex queries, or state transitions (`POST /cart/checkout`, `POST /search`).
- **Appending data:** Adding an entry to a stream, queue, or collection without defining its identity.

Note on Partial Updates: If you need to update only a subset of fields on an existing resource without sending the full payload, use **PATCH** (RFC 5789) rather than PUT.
